

altairsim — Color graphics on a Cromemco Dazzler

2026-09-23 · dbdb362

Li-Chen Wang's **Kaleidoscope** (`KSCOPE`) — the classic Dazzler demo — drawing a four-way-mirrored pattern that wanders and recolors forever.

```
cd examples/dazzler
altairsim kscope.toml
```

The machine comes up **drawing**: `kscope.toml` loads `KSCOPE` and RUNs it from its `startup` list, on an Altair that has a **Dazzler** in it — a Z80 at 4 MHz, 64K of RAM, an 88-2SIO for the console, and the Dazzler's two ports at `0E` / `0F` . On a build with **SDL3** a window opens the moment the Dazzler turns on, and the kaleidoscope appears: a 2 KB, 64×64, 16-color picture.

The processor is a z80 board, at the 4 MHz a Cromemco Z-1 ran. `KSCOPE` itself is 8080 code and does not care — a Z80 runs it unchanged — but it is the CPU the rest of the period Dazzler software wants, and fitting one is a single line in the machine file.

`KSCOPE` never stops on its own (there is no `HLT`), so press **STOP** (`Ctrl-E`) at the terminal to break back to the `altairsim>` prompt; `RUN 0` starts it again. On a **headless** build the machine runs exactly the same and simply draws nothing.

To start it by hand instead — for instance to watch it draw into memory — break out with `STOP` and re-run it yourself:

```
altairsim> LOAD KSCOPE.HEX
loaded 127 bytes (1 page) from KSCOPE.HEX (0000-007E)
altairsim> RUN 0
```

What `KSCOPE` does

The program (`KSCOPE.ASM` , with its assembler listing in `KSCOPE.PRN`) is tiny and assembles at `0000` , so it runs straight from a `RUN 0` :

- It turns the Dazzler **on** with a framebuffer at `0200` (`OUT 0EH` = `81h`), and sets the format to **2 KB, 64×64, color** (`OUT 0FH` = `30h`).
- The 2 KB picture is four 512-byte **quadrants** tiled 2×2. `KSCOPE` draws one pixel and then mirrors it into all four quadrants by negating each axis — which is why the pattern is symmetric about both

the horizontal and vertical center. It walks a pseudo-random path and cycles the color, so the figure is always moving.

Because the framebuffer starts as whatever the RAM powered up holding (random, like real static RAM), the picture emerges from a field of color noise as KSCOPE paints over it.

The Dazzler, briefly

The Dazzler reads its picture straight out of **main memory** — the framebuffer is ordinary RAM (here at `0200`), not on the card. Two `OUT` ports drive it:

- `OUT 0EH` — control: bit 7 on/off, bits 6–0 the high address bits of the framebuffer (so the base is 512-byte aligned).
- `OUT 0FH` — format: resolution ($32\times 32\dots 128\times 128$), size (512 B or 2 KB), color vs 16 greys.

`IN 0EH` reads two status bits (odd/even scan line, end-of-frame) a program can poll to pace its drawing to the frame. See `docs/boards/cromemco-dazzler.md` for the full reference.

Try it yourself

- **Watch it draw into memory.** Break out (`Ctrl-E`) and `DUMP 0200` — the framebuffer bytes KSCOPE has written are right there; each byte is two 4-bit color elements.
- **Change the colors.** The picture is a palette machine: the same bytes look different under a different `OUT 0FH`. `SET dazz0` shows the card; the format is set by the running program.
- **Slow it down or speed it up.** `SET cpu0 clock_hz=2000000` for a 2 MHz Altair, or `clock_hz=0` for flat out.
- **Build this machine yourself.** `recipes/dazzler-from-scratch.pdf` fits every board of it by hand, starting from an empty chassis, and saves the result to a machine file of your own.